

NLP

NLP - Natural Lang processing is a branch of AI that teaches computers to understand, interpret and generate human language (text & speech).

↳ It combines linguistics, computer science and ML to enable applications like chatbots, search engines, translators and voice assistants to process and respond to human communication.

→ NLP Pipeline :-

1) Data Acquisition - collecting the text data.

↳ where you get data from: social media posts, reviews, news articles, chat messages etc.

2) Text cleaning - Removing unwanted noise from text.

↳ like special symbols (@, #, %...), HTML tags, Extra spaces, URLs etc.

3) Text Preprocessing - preparing text in machine-friendly format.

↳ lowercasing → Movie ⇒ movie

Tokenization → splitting into words, ["I", "am", "siddhu"]

stop word removal → removing words like a, the, i

Stemming / Lemmatization → "running" ⇒ "run"

4) Feature Engineering (Vectorization) - converting words into numbers.

5) Model Building - Train the ML model.

↳ Naive Bayes, SVM, Neural Networks, Logistic Reg

6) Evaluation - check how well the model performs.

↳ metrics like accuracy, precision, recall, F1-score etc.

→ Deployment - use the model in real-world application.

Basics

→ Syntax :- syntax deals with how words combine to form grammatical structures.

NLP systems use syntactic rules and probabilistic grammars to analyze sentence structure.

↳ Tasks - tokenization, part-of-speech tagging, chunking, parsing

→ Semantics - focuses on the meaning at word and sentence level.

↳ includes lexical meaning, compositional meaning, semantic relations, resolving ambiguity.

↳ Tasks - word sense disambiguation, semantic role labeling,

Named Entity Recognition, knowledge graph for meaning representation (or embedding)

-> Pragmatics - considers intended meaning based on context, speaker goals, and real-world knowledge.

✱ It handles conversational cues, implied meaning, discourse flow.

(-> Task - dialogue management, coreference resolution, speech act clarification, discourse coherence modeling across multi-sentence text.

Summary :-

1) Syntax - Structure or grammar of a sentence

Eg -> I am Sindhu + ✓

Sindhu I am x wrong syntax

Tasks -> 1) Parts of speech tagging -> noun, verb, adj

2) Parsing - finding sentence structure

3) Grammar checking.

2) Semantics - actual meaning of words & sentence.

Eg -> He is looking for a bank (bank -> river bank / money bank)

Major Tasks -> 1) word sense disambiguation (choosing correct words; meaning)

2) Named Entity Recognition (names, places etc)

3) Semantic role labelling (who did what to whom).

3) Pragmatics -> meaning based on context or situation.

Eg -> I don't like it here.

Major Tasks -> Sentiment Analysis, Dialogue systems (chatbots)

coreference resolution (he, she, it - refers to who?)

Intent detection.

Issues in NLP

NLP faces many challenges because human language is complex, ambiguous and constantly changing.

Major Types :-

1) Linguistic & Semantic Issues :- come from how humans use language

a) Ambiguity - sentences often have multiple interpretations.

↳ Lexical Ambiguity -> One word - multiple meanings.

Eg:- bank - a financial institution / river edge

↳ Syntactic - confusion caused by sentence structure.

Eg:- I saw a boy with binoculars.

↳ Who has binoculars?? (I / he)?

b) ~~Sarcasm~~ Irony → intended meaning - opposite to words.

NLP models struggle to detect - as they often lack access to non-verbal cues like tone or facial expressions.

Eg:- Great! Another exam again! - actually negative.

c) Contextual Dependence → meaning depends on context
(say 90% on context & 10% on words)

Eg → Can you park it?

↳ What's "it" - depends on context.

↳ models often lose track / forget long context.

2) Data Related challenges - Probs with data used to train models.

a) Quality & Noise - Real-world data is noisy - spelling mistakes, symbols, poor grammar, slang → can lead to inaccurate analysis.

b) Linguistic Diversity - Some languages have very little data → low-resource languages.

c) Unstructured Data - Most text come from social media, emails etc.
↳ not in clean format → needs heavy preprocessing.

3) Ethical & Algorithmic Issues :- Probs related to fairness and safety.

a) Bias (Innate Bias) - Models often inherit and amplify societal biases (gender, race, age) present in training data → leads to unfair outputs.

Eg → Doctor - male, Nurse - female.

b) Privacy & Security - NLP uses chats, medical records, personal messages → Risk: data leakage!

c) Computational costs - Needs huge GPUs / TPUs - very expensive - high power consumption.

4) Operational Issues / Barriers :-

a) Real time processing - high accuracy & faster speed for live applications - voice assistants (Siri, Alexa).

b) Constant evolution - Long changes daily $\rightarrow$ new slang, new words, new names.
 $\hookrightarrow$ models must be retrained often.

5) Indian language challenges :-

Indian lang - multiple scripts + rich morphology, code-switching, very less training data (annotated corpora) - make NLP difficult.

Applications of NLP :-

- 1) Sentiment Analysis - Amazon, Swiggy, Twitter
- 2) Voice Assistants & Chatbots - Siri, Alexa, Google Assist, Bank customer support bots
- 3) Translations - Google translator, MS translator.
- 4) Text summarization - News summary, legal doc summarization.
- 5) Information Retrieval - Google search ranking
- 6) Spam & Fraud detection - Gmail, SMS
- 7) Speech to Text / Text-to-speech - YT auto-captions, multilingual transcription generation
- 8) Question Answering systems - ChatGPT, customer service Q/A portals
- 9) Document classification - news-article auto tagging, Email categorization
- 10) Named Entity Recognition - KYC form extraction, resume parsing system

Language Model - It helps in predicting the next word in a sentence.

$\hookrightarrow$ A lang model assigns prob to a sequence of words.

$\hookrightarrow$ Used in speech recognition, translation, text prediction.

Goal: Estimate $P(w_1, w_2, \dots, w_n)$ for a sentence.

A/c to chain rule of prob,

$$P(w_1, w_2, \dots, w_n) = P(w_1) P(w_2|w_1) P(w_3|w_1, w_2) \dots P(w_n|w_1, \dots, w_{n-1})$$

$$\Rightarrow P(w_1, \dots, w_n) = \prod_i P(w_i, \dots, w_{i-1})$$

Each word depends on all prev words (accurate but not practical for computation)

$\rightarrow$ Markov Assumption -

Assumes next word depends only on limited prev words.

$\hookrightarrow$ Reduces dependency to last / prev $(N-1)$ words.

It makes prob estimation feasible.

→ N-gram Language Models:-

1) unigram / 1-gram → No history.

Each word independent - no conditional.

↳ easiest / fastest in terms of calculation

⇒ $P(w_i)$ is considered.

2) Bigram / 2-gram - 1 word history.

↳ $P(w_i | w_{i-1})$

3) Trigram / 3-gram - 2 word history

↳ $P(w_i | w_{i-2}, w_{i-1})$

----- N-gram → n-1 word history.

Higher N gives better context but needs more data.

→ Maximum Likelihood Estimation (MLE) - estimates probability using frequency counts.

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

↳ Based purely on observed data (corpus) ^{training}

↳ simple & mostly used.

!! Problems with MLE → If a word pair never appears, prob = zero

makes whole sentence probability zero !! (because product(π))

Training data is limited - so data sparsity (empty or very rare)

To avoid smoothing is used!

→ Smoothing → a technique used to handle zero probabilities in text data.

↳ Needed - to avoid zero prob for unseen events

Eg → I love NLP

I love coding.

Sentence → I love AI

Since AI never appeared → $P(AI) = 0$
 $P(\text{whole}) = 0$. X

- ↳ To distribute some prob to rare or unseen events.
- ↳ To take prob from words that appear many times (higher count) and give a small amount to words that never appeared (zero count).

1) Laplace (Add-one) Smoothing -

- ↳ Adds 1 to all frequency counts.
- ↳ ensures no prob is zero.

$$P = \frac{(\text{Count} + 1)}{(\text{Total count} + \text{vocabulary size})}$$

↳ Not very accurate for large vocabularies.

2) Good-Turing Smoothing :- method used to estimate probabilities of unseen words/events by using info from rare events (low freq).

eg → People who came twice → help estimate those who came once.

Frequency of Frequencies → N_1 - no. of words occurring once
 N_2 - twice - - -

count c is adjusted with,

$$c^* = \frac{(c+1) N_{c+1}}{N_c}$$

- ↳ Good Turing is more effective than Laplacian smoothing - gives more realistic estimates based on real data distribution.

Terms →

N_c = no. of n -grams (seq of words) that appear c times exactly

singletons → N_1 - only once.

Estimating unseen (N_0) - never appeared.

- ↳ GT used N_1 to estimate how many N -grams might be ~~in~~ in unseen bin (N_0).

↳ Prob P_{GT}^* for things that had zero count N_0 ,

$$P_{GT}^* (\text{freq zero in training}) = \frac{N_1}{N}$$

$$P_{GT}^v \text{ (for non-zero freq)} = \frac{C_k}{N}$$

Eg:- $V = \{ \text{Apple, Banana, Cherry, Date, Fig, Grape, Jackfruit, Mango} \}$

Training corpus $\rightarrow$ Mango, cherry, cherry, cherry, cherry, cherry,
Banana, Banana, Jackfruit, Grapes, Fig, Mango, Banana, cherry, cherry,
cherry, cherry, cherry.

Ans $\rightarrow$ word - freq

cherry - 10

Banana - 3

Mango - 2

Fig - 1

Grapes - 1

Jackfruit - 1

unseen words $\rightarrow$ Apple, Date.

$N_1 \rightarrow 3$ (Fig, Grapes, JF)

$N_2 \rightarrow 1$ (Mango)

$N_3 \rightarrow 1$ (Banana)

$N_{10} \rightarrow 1$ (cherry) $N_0 = ??$

	N_0	N_1	N_2	N_3	N_{10}
count	2 (Apple, Date)	3	1	1	1
MLE(P)	0/18 = 0	3/18 = 0.06	2/18 = 0.11	3/18	10/18
C_k	-	$2 \left(\frac{1}{3} \right) = 0.67$	$3 \left(\frac{1}{1} \right) = 3$	4 $\frac{1}{(N_0 N_4)}$	$\frac{1}{(N_0 N_{11})}$
P_{GT}	0.085 N/N $\Rightarrow 3/18 = 0.17$	$0.67/18 \left(\frac{C_k}{N} \right)$	$3/18$	-	-
Apple	$0.17/2 = 0.085$	-	-	-	-
Date	0.085	-	-	-	-

$\Rightarrow$ Backoff & Interpolation $\rightarrow$

$\rightarrow$ Backoff - If trigram is unseen $\rightarrow$ use bigram/unigram.
 $\hookrightarrow$ uses higher order model when available.

$$P(w_i | w_{i-2} w_{i-1}) = \begin{cases} P(w_i | w_{i-2} w_{i-1}) & \text{if } c(w_i | w_{i-2} w_{i-1}) > 0 \\ \lambda_1 P(w_i | w_{i-1}) & \text{if } c(w_i | w_{i-2} w_{i-1}) = 0 \\ & \text{ \& if } (c(w_i | w_{i-1}) > 0) \\ \lambda_2 P(w_i) & \text{otherwise.} \end{cases}$$

Here $\lambda_1 + \lambda_2 = 1$ (always)

2) Interpolation $\rightarrow$ combined unigram, bigram and trigram (additive)

$$P(w_i | w_{i-2}, w_{i-1}) = d_1 P(w_i | w_{i-2}, w_{i-1}) + d_2 P(w_i | w_{i-1}) + d_3 P(w_i)$$

Conditions,
 $d_1 + d_2 + d_3 = 1$

All d values $\geq 0 \Rightarrow d_1 \geq 0, d_2 \geq 0, d_3 \geq 0$

$\rightarrow$ Perplexity $\rightarrow$ measures how well a language model can predict a sentence.

$\hookrightarrow$ tells how confused the model is while predicting

Low perplexity $\rightarrow$ model is less confused $\rightarrow$ good/better model

Higher perp $\rightarrow$ more confused $\rightarrow$ bad model.

n-gram $\Rightarrow P = \left(\frac{1}{P(w_1, w_2, \dots, w_n)} \right)^{1/N} \quad (21) \quad N \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_n)}}$

For Bigrams,

$$PP(w) = N \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing Perplexity $\Leftrightarrow$ Maximizing Probability

Probability,

$$P(s) = 1$$

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Independent events $\rightarrow P(A \cap B) = P(A)P(B)$

Bayes theorem $\Rightarrow P(B|A) = \frac{P(A|B)P(B)}{P(A)}$

$$\left[\begin{array}{l} P(A \cap B) = \\ P(A|B)P(B) \end{array} \right]$$